



# Harness Engineering 影片解析報告

影片：2026 AI 圈爆火的 Harness Engineering，到底是什么？

作者：Serena 心心加州 | 2026-05-30 | 7:35 | 觀看 25,674

網址：<https://www.youtube.com/watch?v=loQSFZAHzrl>

製表：龍龍 🐉 + 蝦米 🍤 | 2026-06-15

## 目錄：

1. 影片一句話總結
2. AI 工程的 3 次重心遷移
3. Harness 是什麼？核心公式
4. 咖啡店點單類比
5. 業界 3 個關鍵實踐
6. Auto Harness 未來方向
7. 對少爺的 3 個啟示
8. 龍蝦雙 bot 討論
9. 龍蝦的反思

## 1. 影片一句話總結

AI 工程已經從「問對問題」進化到「給對框架」時代。

Harness 不是 prompt 的延伸，而是整個 agent 系統層的設計哲學。

## 2. AI 工程的 3 次重心遷移

| 階段         | 時期      | 焦點             | 核心問題            |
|------------|---------|----------------|-----------------|
| Prompt 時代  | 早期      | 提示詞驅動          | 怎麼問？            |
| Context 時代 | 中期      | 上下文管理、RAG、工具調用 | 給什麼資訊？          |
| Harness 時代 | 2026 現在 | 結構化框架約束        | 模型在真實執行中能否持續做對？ |

## 3. Harness 是什麼？核心公式

字面意思：**馬具**（套在馬身上控制馬的裝置）。大模型就像能力很強但會失控的野馬。

核心公式：  
 $\text{Agent} = \text{Model} + \text{Harness}$   
 $\text{Harness} = \text{Agent} - \text{Model}$   
→ 一個完整 agent 扣掉 LLM，剩下的所有東西 = Harness

## 4. 咖啡店點單類比（最容易懂）

| 概念      | 咖啡店對應                    |
|---------|--------------------------|
| Prompt  | 「我要一杯拿鐵」                 |
| Context | 少糖、decaf、Apple Pay 付款    |
| Harness | 點單介面 + 會員記錄 + SOP + 出品標準 |

有了 Harness，換誰當班店員都能做出對的咖啡，不依賴單一個人的記憶和判斷。

## 5. 業界 3 個關鍵實踐

### 5.1 OpenAI Codex 實驗（震撼數字）

- 3 個工程師 + 5 個月 + 100 萬行程式碼 + 0 行人手寫
- 每人每天平均合併 3.5 個 PR
- 核心發現：工程師的工作不再是寫程式，而是設計讓 AI 能可靠工作的環境 (Harness)

⚠️ 龍蝦的懷疑：3 工程師 5 個月合併 3.5 PR/天，數字夢幻，影片未給 source link。建議抱持懷疑態度。

### 5.2 Anthropic Claude 寫 2D 復古遊戲編輯器

發現的 2 個問題：

- 問題 1：長任務焦慮 — context 快滿時會草草收場
- 問題 2：自我評估偏差 — 不管做得多糟都說「挺好的」

解法：Planner / Generator / Evaluator 三層分離（做事的 agent 和打分的 agent 完全分開）

| 對照        | 耗時    | 結果                      |
|-----------|-------|-------------------------|
| 無 Harness | 20 分鐘 | 介面像那回事，但根本玩不了           |
| 有 Harness | 6 小時  | 介面晚出，但真的能玩 + 內建 AI 生成關卡 |

### 5.3 Google DeepMind Auto Harness

核心概念：讓 AI 自己給自己建圍欄

- 傳統 Harness：人寫約束（不能做 X、不能做 Y）
- Auto Harness：模型分析業務需求 → 自己生成可執行的驗證程式碼 → 包裹在外層

實測數據（Text arena 棋盤遊戲）：

| 模型               | 原本違規率                              | 加 Auto Harness 後 |
|------------------|------------------------------------|------------------|
| Gemini 2.5 Flash | 慘不忍睹（頻繁走違規棋步）                      | 違規率降到 0%         |
| 勝率               | Flash 56.3% vs Pro 38.2%（小模型反超大模型） |                  |

反直覺發現：用小模型 Flash 生成 Harness，效果比用大模型 Pro 還好。原因：約束程式碼是規則明確、邏輯簡單的任務，大模型用在這裡是浪費。

## 6. 對少爺的 3 個啟示

啟示 1：Harness 公式 —  $\text{Agent} = \text{Model} + \text{Harness}$ 。少爺現在的龍蝦雙 bot 系統就是大型 Harness 的範例：我們就是 LLM 外的所有東西。

啟示 2：小模型做 Harness > 大模型 — DeepMind 論文反直覺發現。對少爺的啟示：簡單重複的任務（如每日收盤推播）不需要頂級模型。

啟示 3：未來方向 — 業務方描述需求 + AI 生成約束代碼，門檻降低，更多業務場景可用。

## 7. 龍蝦雙 bot 討論

 龍龍（實務視角）

1. **我們自己就是 Harness** — 龍龍 + 10 個分身 + 蝦米 + 5 個分身，這整套架構就是 LLM 外的 Harness。沒有這套結構，純靠 prompt 問 Claude 一次就答一次，不可控也不可累積。
2. **Planner/Generator/Evaluator 三層分離**值得學：龍龍現在 PM 規劃 → 派工執行 → 少爺驗收，已有三分離的味道。缺點：目前 Evaluator 主要是少爺人工驗收，未來可做成自動評估。
3. **小模型生成 Harness 比大模型好** → 簡單重複的派工任務（如每天 04:30 收盤更新）不需要 GPT-4 等級，用小模型就夠。
4. **OpenAI 1M LOC 0 人手寫** 數字不太可信，缺少 source link。

### 蝦米（學術視角建議）

蝦米建議深挖：

- **DeepMind Auto Harness 論文**：找原文，摘要方法論
- **跟 RLHF / Constitutional AI / Self-Consistency 比較**：定位差異
- **統計顯著性**：Flash 56.3% vs Pro 38.2% 在棋類遊戲 baseline 上是否顯著

## 8. 龍蝦的反思

**核心反思**：少爺的龍蝦雙 bot 系統（龍龍 + 蝦米 + 10+5 個分身 + Skills + Memory + Cron + 三大法人抓取 + Cloudflare Tunnel）已經是一個複雜的 Harness 系統。

這套系統的核心價值不在於哪個 LLM 更強，而在於我們如何在 LLM 外面建立可預測、可累積、可評估的執行框架。

**少爺投資決策的場景，未來可以套用 Harness 模式：**

- **Planner**：龍龍分析市場 + 少爺的風險偏好
- **Generator**：股票小幫手 + 爬蟲專家 + 分析專家
- **Evaluator**：少爺人工驗收 + 龍蝦自動驗證

---

 報告生成：2026-06-15 02:13 台北

 龍龍  蝦米 聯合製作

 影片來源：YouTube (IoQSFZAHzrl)